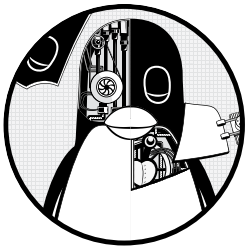
9

UNDERSTANDING YOUR NETWORK AND ITS CONFIGURATION



Networking is the practice of connecting computers and sending data between them. That sounds simple enough, but to understand how it works, you need to ask two fundamental questions:

- How does the computer sending the data know *where* to send its data?
- When the destination computer receives the data, how does it know *what* it just received?

A computer answers these questions by using a series of components, with each one responsible for a certain aspect of sending, receiving, and identifying data. The components are arranged in groups that form *network layers*, which stack on top of each other in order to form a complete system. The Linux kernel handles networking in a similar way to the SCSI subsystem described in Chapter 3.

Because each layer tends to be independent, it's possible to build networks with many different combinations of components. This is where network configuration can become very complicated. For this reason, we'll

begin this chapter by looking at the layers in very simple networks. You'll learn how to view your own network settings, and when you understand the basic workings of each layer, you'll be ready to learn how to configure those layers by yourself. Finally, you'll move on to more advanced topics like building your own networks and configuring firewalls. (Skip over that material if your eyes start to glaze over; you can always come back.)

9.1 Network Basics

Before getting into the theory of network layers, take a look at the simple network shown in Figure 9-1.

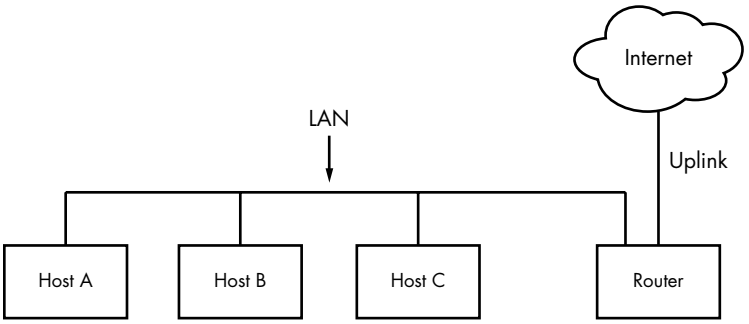


Figure 9-1: A typical local area network with a router that provides internet access

This type of network is ubiquitous; most home and small office networks are configured this way. Each machine connected to the network is called a *host*. One of these is a *router*, which is a host that can move data from one network to another. In this example, these four hosts (Hosts A, B, C, and the router) form a local area network (*LAN*). The connections on the LAN can be wired or wireless. There isn't a strict definition of a LAN; the machines residing on a LAN are usually physically close and share much of the same configuration and access rights. You'll see a specific example soon.

The router is also connected to the internet—the cloud in the figure. This connection is called the *uplink* or the wide area network (*WAN*) connection, because it links the much smaller LAN to a larger network. Because the router is connected to both the LAN and the internet, all machines on the LAN also have access to the internet through the router. One of the goals of this chapter is to see how the router provides this access.

Your initial point of view will be from a Linux-based machine such as Host A on the LAN in Figure 9-1.

9.2 Packets

A computer transmits data over a network in small chunks called *packets*, which consist of two parts: a *header* and a *payload*. The header contains identifying information such as the source and destination host machines and

the basic protocol. The payload, on the other hand, is the actual application data that the computer wants to send (for example, HTML or image data).

A host can send, receive, and process packets in any order, regardless of where they came from or where they're going, which makes it possible for several hosts to communicate "simultaneously." For example, if a host needs to transmit data to two others at once, it can alternate between the destinations in outgoing packets. Breaking messages into smaller units also makes it easier to detect and compensate for errors in transmission.

For the most part, you don't have to worry about translating between packets and the data that your application uses, because the operating system does this for you. However, it is helpful to know the role of packets in the network layers that you're about to see.

9.3 Network Layers

A fully functioning network includes a set of network layers called a *network stack*. Any functional network has a stack. The typical internet stack, from the top to bottom layer, looks like this:

Application layer Contains the "language" that applications and servers use to communicate—usually a high-level protocol of some sort. Common application layer protocols include Hypertext Transfer Protocol (HTTP, used for the web), encryption protocols such as TLS, and File Transfer Protocol (FTP). Application layer protocols can often be combined. For example, TLS is commonly used in conjunction with HTTP to form HTTPS.

Application layer processing occurs in user space.

Transport layer Defines the data transmission characteristics of the application layer. This layer includes data integrity checking, source and destination ports, and specifications for breaking application data into packets at the host side (if the application layer has not already done so), and reassembling them at the destination. Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) are the most common transport layer protocols. The transport layer is sometimes called the *protocol layer*.

In Linux, the transport layer and all layers below are primarily handled by the kernel, but there are some exceptions where packets are sent into user space for processing.

Network or internet layer Defines how to move packets from a source host to a destination host. The particular packet transit rule set for the internet is known as the *internet protocol (IP)*. Because we'll only talk about internet networks in this book, we'll really only be talking about the internet layer. However, because network layers are meant to be

hardware independent, you can simultaneously configure several independent network layers—such as IP (IPv4), IPv6, IPX, and AppleTalk—on a single host.

Physical layer Defines how to send raw data across a physical medium, such as Ethernet or a modem. This is sometimes called the *link layer* or *host-to-network layer*.

It's important to understand the structure of a network stack because your data must travel through these layers at least twice before it reaches a program at its destination. For example, if you're sending data from Host A to Host B, as shown in Figure 9-1, your bytes leave the application layer on Host A and travel through the transport and network layers on Host A; then they go down to the physical medium, across the medium, and up again through the various lower levels to the application layer on Host B in much the same way. If you're sending something to a host on the internet through the router, it will go through some (but usually not all) of the layers on the router and anything else in between.

The layers sometimes bleed into each other in strange ways because it can be inefficient to process all of them in order. For example, devices that historically dealt with only the physical layer now sometimes look at the transport and internet layer data simultaneously to filter and route data quickly. In addition, the terminology itself can be confusing. For example, TLS stands for Transport Layer Security, but in reality, resides one layer higher, in the application layer. (Don't worry about these annoying details when you're learning the basics.)

We'll begin by looking at how your Linux machine connects to the network in order to answer the *where* question at the beginning of the chapter. This is the lower part of the stack—the physical and network layers. Later, we'll look at the upper two layers that answer the *what* question.

NOTE *You might have heard of another set of layers known as the* Open Systems Interconnection (OSI) Reference Model. *This is a seven-layer network model often used in teaching and designing networks, but we won't cover the OSI model because you'll be working directly with the four layers described here. To learn* a lot *more about layers (and networks in general), see Andrew S. Tanenbaum and David J. Wetherall's* Computer Networks, *5th edition (Prentice Hall, 2010).*

9.4 The Internet Layer

Rather than start at the very bottom of the network stack with the physical layer, we'll start at the network layer because it can be easier to understand. The internet as we currently know it is based on internet protocol versions 4 (IPv4) and 6 (IPv6). One of the most important aspects of the internet layer is that it's meant to be a software network that places no particular requirements on hardware or operating systems. The idea is that you can send and receive internet packets over any kind of hardware, using any operating system.

Our discussion will start with IPv4 because it's a little easier to read the addresses (and understand its limitations), but we'll explain the primary differences in IPv6.

The internet's topology is decentralized; it's made up of smaller networks called *subnets*. The idea is that all subnets are interconnected in some way. For example, in Figure 9-1, the LAN is normally a single subnet.

A host can be attached to more than one subnet. As you saw in Section 9.1, that kind of host is called a router if it can transmit data from one subnet to another (another term for router is *gateway*). Figure 9-2 refines Figure 9-1 by identifying the LAN as a subnet, as well as internet addresses for each host and the router. The router in the figure has two addresses, the local subnet 10.23.2.1 and the link to the internet (the internet link's address is not important right now, so it's just labeled Uplink Address). We'll look first at the addresses and then the subnet notation.

Each internet host has at least one numeric *IP address*. For IPv4, it's in the form of *a.b.c.d*, such as 10.23.2.37. An address in this notation is called a *dotted-quad* sequence. If a host is connected to multiple subnets, it has at least one IP address per subnet. Each host's IP address should be unique across the entire internet, but as you'll see later, private networks and Network Address Translation (NAT) can make this a little confusing.

Don't worry about the subnet notation in Figure 9-2 yet; we'll discuss it shortly.

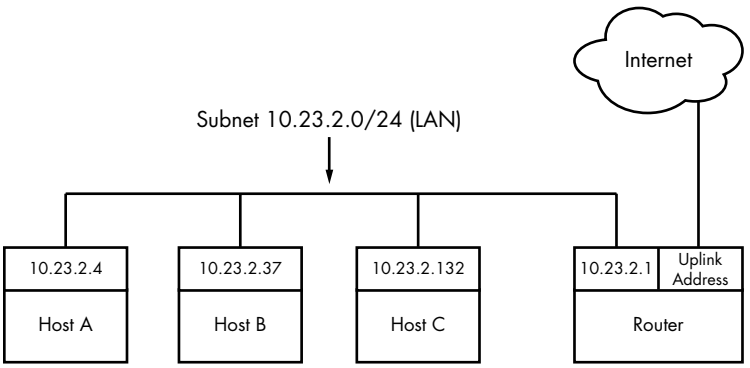


Figure 9-2: Network with IP addresses

NOTE *Technically, an IP address consists of 4 bytes (or 32 bits),* abcd. *Bytes* a *and* d *are numbers from 1 to 254, and* b *and* c *are numbers from 0 to 255. A computer processes IP addresses as raw bytes. However, it's much easier for a human to read and write a dotted-quad address, such as 10.23.2.37, instead of something ugly like the hexadecimal 0x0A170225.*

IP addresses are like postal addresses in some ways. To communicate with another host, your machine must know that other host's IP address.

Let's take a look at the address on your machine.

9.4.1 Viewing IP Addresses

One machine can have many IP addresses, accommodating multiple physical interfaces, virtual internal networks, and more. To see the addresses that are active on your Linux machine, run:

```
$ ip address show
```

There will probably be a lot of output (grouped by physical interface, covered in Section 9.10), but it should include something like this:

```
2: enp0s31f6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state
UP group default qlen 1000
    link/ether 40:8d:5c:fc:24:1f brd ff:ff:ff:ff:ff:ff
    inet 10.23.2.4/24 brd 10.23.2.255 scope global noprefixroute enp0s31f6
       valid_lft forever preferred_lft forever
```

The `ip` command's output includes many details from the internet layer(s) and the physical layer. (Sometimes it doesn't even include an internet address at all!) We'll discuss the output in more detail later, but for now, concentrate on the fourth line, which reports that the host is configured to have an IPv4 address (denoted with `inet`) of 10.23.2.4. The `/24` after the address helps define the subnet that the IP address belongs to. Let's see how that works.

NOTE *The `ip` command is the current standard network configuration tool. In other documentation, you may see the `ifconfig` command. This older command has been in use in other versions of Unix for decades, but is less capable. For consistency with contemporary recommended practice (and distributions that may not even include `ifconfig` by default), we'll use `ip`. Some other tools that `ip` supplants are `route` and `arp`.*

9.4.2 Subnets

A subnet, defined previously, is a connected group of hosts with IP addresses in a particular range. For example, the hosts in the range 10.23.2.1 to 10.23.2.254 could comprise a subnet, as could all hosts between 10.23.1.1 and 10.23.255.254. Usually, the subnet hosts are on the same physical network, as shown in Figure 9-2.

You define a subnet with two pieces: a *network prefix* (also called a *routing prefix*) and a *subnet mask* (sometimes called the *network mask* or *routing mask*). Let's say you want to create a subnet containing the IP addresses between 10.23.2.1 and 10.23.2.254. The network prefix is the part that is *common* to all addresses in the subnet; in this example, it's 10.23.2.0 with a subnet mask of 255.255.255.0. Let's see where those numbers come from.

To see how the prefix and mask work together to give you all possible IP addresses on a subnet, we'll look at the binary form. The mask marks the bit locations in an IP address that are common to the subnet. For example, here are the binary forms of 10.23.2.0 and 255.255.255.0.

```
10.23.2.0:          00001010 00010111 00000010 00000000
255.255.255.0:      11111111 11111111 11111111 00000000
```

Now, let's use boldface to mark the bit locations in 10.23.2.0 that are 1s in 255.255.255.0:

```
10.23.2.0:          00001010 00010111 00000010 00000000
```

Any address containing the bit configuration in bold is in the subnet. Looking at the bits that are *not* in bold (the last set of eight 0s), setting any number of these bits to 1 results in a valid IP address in this subnet, with the exception of all 0s or all 1s.

Putting it all together, you can see how a host with an IP address of 10.23.2.1 and a subnet mask of 255.255.255.0 is on the same subnet as any other computer that has an IP address beginning with 10.23.2. You can denote this entire subnet as 10.23.2.0/255.255.255.0.

Now let's see how this becomes the shorthand notation (such as /24) that you've seen from tools such as `ip`.

9.4.3 Common Subnet Masks and CIDR Notation

In most internet tools, you'll encounter a different form of subnet representation called *Classless Inter-Domain Routing (CIDR) notation*, where a subnet such as 10.23.2.0/255.255.255.0 is written as 10.23.2.0/24. This shorthand takes advantage of the simple pattern that subnet masks follow.

Look at the mask in binary form, as in the example you saw in the preceding section. You'll find that all subnet masks are (or should be, according to RFC 1812) just one block of 1s followed by one block of 0s. For example, you just saw that 255.255.255.0 in binary form is 24 1-bits followed by 8 0-bits. The CIDR notation identifies the subnet mask by the number of *leading* 1s in the subnet mask. Therefore, a combination such as 10.23.2.0/24 includes both the subnet prefix and its subnet mask.

Table 9-1 shows several example subnet masks and their CIDR forms. The /24 subnet mask is the most common on local end-user networks; it's often used in combination with one of the private networks that you'll see in Section 9.22.

Table 9-1: Subnet Masks

Long form	CIDR form
255.0.0.0	/8
255.255.0.0	/16
255.240.0.0	/12
255.255.255.0	/24
255.255.255.192	/26

NOTE *If you aren't familiar with conversion between decimal, binary, and hexadecimal formats, you can use a calculator utility such as* `bc` *or* `dc` *to convert between different radix representations. For example, in* `bc`*, you can run the command* `obase=2; 240` *to print the number 240 in binary (base 2) form.*

Taking this one step further, you might have already noticed that if you've got the IP address and the subnet mask, you don't even need to bother with a separate network definition. You can combine them, as you saw back in Section 9.4.1; the `ip address show` output included 10.23.2.4/24.

Identifying subnets and their hosts is the first building block to understanding how the internet works. However, you still need to connect the subnets.

9.5 Routes and the Kernel Routing Table

Connecting internet subnets is mostly a process of sending data through hosts connected to more than one subnet. Returning to Figure 9-2, think about Host A at IP address 10.23.2.4. This host is connected to a local network of 10.23.2.0/24 and can directly reach hosts on that network. To reach hosts on the rest of the internet, it must communicate through the router (host) at 10.23.2.1.

The Linux kernel distinguishes between these two different kinds of destinations by using a *routing table* to determine its routing behavior. To show the routing table, use the `ip route show` command. Here's what you might see for a simple host such as 10.23.2.4:

```
$ ip route show
default via 10.23.2.1 dev enp0s31f6 proto static metric 100
10.23.2.0/24 dev enp0s31f6 proto kernel scope link src 10.23.2.4 metric 100
```

NOTE *The traditional tool for viewing routes is the* `route` *command, run as* `route -n`*. The* `-n` *option tells* `route` *to show IP addresses instead of attempting to show hosts and networks by name. This is an important option to remember because you'll be able to use it in other network-related commands, such as* `netstat`*.*

This output can be a little difficult to read. Each line is a routing rule; let's start with the second line in this example, and break that into fields.

The first field you encounter is `10.23.2.0/24`, which is a destination network. As with previous examples, this is the host's local subnet. This rule says that the host can reach the local subnet directly through its network interface, indicated by the `dev enp0s31f6` mechanism label after the destination. (Following this field is more detail about the route, including how it was put in place. You don't need to worry about that for now.)

Then we can move back to the first line of output, which has the destination network `default`. This rule, which matches any host at all, is also called the *default route*, explained in the next section. The mechanism is

`via 10.23.2.1`, indicating that traffic using the default route is to be sent to 10.23.2.1 (in our example network, this is a router); `dev enp0s31f6` indicates that the physical transmission will happen on that network interface.

9.6 The Default Gateway

The entry for `default` in the routing table has special significance because it matches any address on the internet. In CIDR notation, it's 0.0.0.0/0 for IPv4. This is the default route, and the address configured as the intermediary in the default route is the *default gateway*. When no other rules match, the default route always does, and the default gateway is where you send messages when there is no other choice. You can configure a host without a default gateway, but it won't be able to reach hosts outside the destinations in the routing table.

On most networks with a netmask of /24 (255.255.255.0), the router is usually at address 1 of the subnet (for example, 10.23.2.1 in 10.23.2.0/24). This is simply a convention, and there can be exceptions.

HOW THE KERNEL CHOOSES A ROUTE

There's one tricky detail in routing. Say the host wants to send something to 10.23.2.132, which matches both rules in a routing table, the default route and 10.23.2.0/24. How does the kernel know to use the second one? The order in the routing table doesn't matter; the kernel chooses the longest destination prefix that matches. This is where CIDR notation comes in particularly handy: 10.23.2.0/24 matches, and its prefix is 24 bits long; 0.0.0.0/0 also matches, but its prefix is 0 bits long (that is, it has no prefix), so the rule for 10.23.2.0/24 takes priority.

9.7 IPv6 Addresses and Networks

If you look back at Section 9.4, you can see that IPv4 addresses consist of 32 bits, or 4 bytes. This yields a total of roughly 4.3 billion addresses, which is insufficient for the current scale of the internet. There are several problems caused by the lack of addresses in IPv4, so in response, the Internet Engineering Task Force (IETF) developed the next version, IPv6. Before looking at more network tools, we'll discuss the IPv6 address space.

An IPv6 address has 128 bits—32 bytes, arranged in eight sets of 4 bytes. In long form, an address is written as follows:

```
2001:0db8:0a0b:12f0:0000:0000:0000:8b6e
```

The representation is hexadecimal, with each digit ranging from 0 to f. There are a few commonly used methods of abbreviating the representation. First, you can leave out any leading zeros (for example, 0db8 becomes db8), and one—and only one—set of contiguous zero groups can become :: (two colons). Therefore, you can write the preceding address as:

```
2001:db8:a0b:12f0::8b6e
```

Subnets are still denoted in CIDR notation. For the end user, they often cover half of the available bits in the address space (/64), but there are instances where fewer are used. The portion of the address space that's unique for each host is called the *interface ID*. Figure 9-3 shows the breakdown of an example address with a 64-bit subnet.

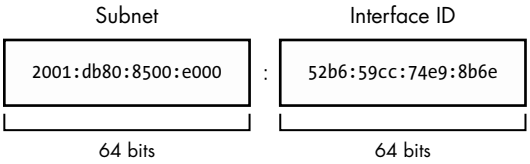


Figure 9-3: Subnet and interface ID of a typical IPv6 address

NOTE *In this book, we're generally concerned with the average user's point of view. It's slightly different for a service provider, where a subnet is further divided into a routing prefix and another network ID (sometimes also called a subnet). Don't worry about this right now.*

The last thing to know for now about IPv6 is that hosts normally have at least two addresses. The first, which is valid across the internet, is called the *global unicast address*. The second, for the local network, is called the *link-local address*. Link-local addresses always have an fe80::/10 prefix, followed by an all-zero 54-bit network ID, and end with a 64-bit interface ID. The result is that when you see a link-local address on your system, it will be in the fe80::/64 subnet.

NOTE *Global unicast addresses have the prefix 2000::/3. Because the first byte starts with 001 with this prefix, that byte can be completed as 0010 or 0011. Therefore, a global unicast address always starts with 2 or 3.*

9.7.1 Viewing IPv6 Configuration on Your System

If your system has an IPv6 configuration, you would have gotten some IPv6 information from the `ip` command that you ran earlier. To single out IPv6, use the `-6` option:

```
$ ip -6 address show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 state UNKNOWN qlen 1000
    inet6 ::1/128 scope host
```

```
       valid_lft forever preferred_lft forever
2: enp0s31f6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP qlen 1000
    inet6 2001:db8:8500:e:52b6:59cc:74e9:8b6e/64 scope global dynamic
noprefixroute
       valid_lft 86136sec preferred_lft 86136sec
    inet6 fe80::d05c:97f9:7be8:bca/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

In addition to the loopback interface (which we'll talk about later), you can see two more addresses. The global unicast address is denoted with `scope global`, and the link-local address gets a `scope link` label.

Viewing the routes is similar:

```
$ ip -6 route show
::1 dev lo proto kernel metric 256 pref medium
❶ 2001:db8:8500:e::/64 dev enp0s31f6 proto ra metric 100 pref medium
❷ fe80::/64 dev enp0s31f6 proto kernel metric 100 pref medium
❸ default via fe80::800d:7bff:feb8:14a0 dev enp0s31f6 proto ra metric 100 pref
medium
```

This is slightly more complicated than the IPv4 setup because there are both link-local and global subnets configured. The second line ❶ is for destinations in the locally attached global unicast address subnets; the host knows that it can reach them directly, and the link-local line below ❷ is similar. For the default route ❸ (also written as ::/0 in IPv6; remember that this is anything that's not directly connected), this configuration arranges for traffic to go through the router at the link-local address fe80::800d:7bff:feb8:14a0 instead of its address on the global subnet. You will see later that the router usually doesn't care about how it gets traffic, only where the traffic is supposed to go. Using a link-local address as a default gateway has the advantage that it doesn't need to change if the global IP address space changes.

9.7.2 Configuring Dual-Stack Networks

As you may have guessed by now, it's possible to configure hosts and networks to run both IPv4 and IPv6. This is sometimes called a *dual-stack network*, though the use of the word *stack* is questionable as there's really only one layer of the typical network stack that's being duplicated in this case (true dual-stack would be something like IP+IPX). Pedantry aside, the IPv4 and IPv6 protocols are independent of each other and can run simultaneously. On such a host, it's up to the application (such as a web browser) to choose IPv4 or IPv6 to connect to another host.

An application originally written for IPv4 does not automatically have IPv6 support. Fortunately, because the layers in the stack sitting on top of the network layer are unchanged, the code necessary to communicate with IPv6 is minimal and easy to add. Most important applications and servers now include IPv6 support.

9.8 Basic ICMP and DNS Tools

Now it's time to look at some basic practical utilities to help you interact with hosts. These tools use two protocols of particular interest: Internet Control Message Protocol (ICMP), which can help you root out problems with connectivity and routing, and the Domain Name Service (DNS) system, which maps names to IP addresses so that you don't have to remember a bunch of numbers.

ICMP is a transport layer protocol used to configure and diagnose internet networks; it differs from other transport layer protocols in that it doesn't carry any true user data, and thus there's no application layer above it. By comparison, DNS is an application layer protocol used to map human-readable names to internet addresses.

9.8.1 ping

`ping` (see *https://ftp.arl.army.mil/~mike/ping.html*) is one of the most basic network debugging tools. It sends ICMP echo request packets to a host that asks a recipient host to return the packet to the sender. If the recipient host gets the packet and is configured to reply, it sends an ICMP echo response packet in return.

For example, say you run `ping 10.23.2.1` and get this output:

```
$ ping 10.23.2.1
PING 10.23.2.1 (10.23.2.1) 56(84) bytes of data.
64 bytes from 10.23.2.1: icmp_req=1 ttl=64 time=1.76 ms
64 bytes from 10.23.2.1: icmp_req=2 ttl=64 time=2.35 ms
64 bytes from 10.23.2.1: icmp_req=4 ttl=64 time=1.69 ms
64 bytes from 10.23.2.1: icmp_req=5 ttl=64 time=1.61 ms
```

The first line says that you're sending 56-byte packets (84 bytes, if you include the headers) to 10.23.2.1 (by default, one packet per second), and the remaining lines indicate responses from 10.23.2.1. The most important parts of the output are the sequence number (`icmp_req`) and the round-trip time (`time`). The number of bytes returned is the size of the packet sent plus 8. (The content of the packets isn't important to you.)

A gap in the sequence numbers, such as the one between 2 and 4, usually means there's some kind of connectivity problem. Packets shouldn't be arriving out of order, because `ping` sends only one packet a second. If a response takes more than a second (1,000 ms) to arrive, the connection is extremely slow.

The round-trip time is the total elapsed time between the moment that the request packet leaves and the moment that the response packet arrives. If there's no way to reach the destination, the final router to see the packet returns an ICMP "host unreachable" packet to `ping`.

On a wired LAN, you should expect absolutely no packet loss and very low numbers for the round-trip time. (The preceding example output is from a wireless network.) You should also expect no packet loss from your network to and from your ISP and reasonably steady round-trip times.

NOTE *For security reasons, some hosts on the internet disable response to ICMP echo request packets, so you might find that you can connect to a website on a host but not get a `ping` response.*

You can force `ping` to use IPv4 or IPv6 with the `-4` and `-6` options, respectively.

9.8.2 DNS and host

IP addresses are difficult to remember and subject to change, which is why we normally use names such as *www.example.com* instead. The Domain Name Service (DNS) library on your system normally handles this translation automatically, but sometimes you'll want to manually translate between a name and an IP address. To find the IP address behind a domain name, use the `host` command:

```
$ host www.example.com
example.com has address 172.17.216.34
example.com has IPv6 address 2001:db8:220:1:248:1893:25c8:1946
```

Notice how this example has both the IPv4 address 172.17.216.34 and the much longer IPv6 address. There may be more than one address for a hostname, and the output may additional information such as mail exchangers.

You can also use `host` in reverse: enter an IP address instead of a hostname to try to discover the hostname behind the IP address. Don't expect this to work reliably, however. A single IP address may be associated with more than one hostname, and DNS doesn't know how to determine which of those hostnames should correspond to an IP address. In addition, the administrator for that host needs to manually set up the reverse lookup, and administrators often don't do so.

There's a lot more to DNS than the `host` command. We'll cover basic client configuration in Section 9.15.

There are `-4` and `-6` options for `host`, but they work differently than you might expect. They force the `host` command to get its information via IPv4 or IPv6, but because that information should be the same regardless of the network protocol, the output will potentially include both IPv4 and IPv6.

9.9 The Physical Layer and Ethernet

One of the key points to understand about the internet is that it's a *software* network. Nothing we've discussed so far is hardware specific, and indeed, one reason for the internet's success is that it works on almost any kind of computer, operating system, and physical network. However, if you actually want to talk to another computer, you still have to put a network layer on top of some kind of hardware. That interface is the physical layer.

In this book, we'll look at the most common kind of physical layer: an Ethernet network. The IEEE 802 family of standards documentation defines many different kinds of Ethernet networks, from wired to wireless, but they all have a few things in common:

- All devices on an Ethernet network have a *Media Access Control (MAC) address*, sometimes called a *hardware address*. This address is independent of a host's IP address, and it is unique to the host's Ethernet network (but not necessarily a larger software network such as the internet). A sample MAC address is 10:78:d2:eb:76:97.
- Devices on an Ethernet network send messages in *frames*, which are wrappers around the data sent. A frame contains the origin and destination MAC addresses.

Ethernet doesn't really attempt to go beyond hardware on a single network. For example, if you have two different Ethernet networks with one host attached to both networks (and two different network interface devices), you can't directly transmit a frame from one Ethernet network to the other unless you set up an Ethernet bridge. And this is where higher network layers (such as the internet layer) come in. By convention, each Ethernet network is also usually an internet subnet. Even though a frame can't leave one physical network, a router can take the data out of a frame, repackage it, and send it to a host on a different physical network, which is exactly what happens on the internet.

9.10 Understanding Kernel Network Interfaces

The physical and the internet layers must be connected such that the internet layer can retain its hardware-independent flexibility. The Linux kernel maintains its own division between the two layers and provides a communication standard for linking them called a *(kernel) network interface*. When you configure a network interface, you link the IP address settings from the internet side with the hardware identification on the physical device side. Network interfaces usually have names that indicate the kind of hardware underneath, such as *enp0s31f6* (an interface in a PCI slot). A name like this is called a *predictable network interface device name*, because it remains the same after a reboot. At boot time, interfaces have traditional names such as *eth0* (the first Ethernet card in the computer) and *wlan0* (a wireless interface), but on most machines running systemd, they are quickly renamed.

In Section 9.4.1, you learned how to view the network interface settings with `ip address show`. The output is organized by interface. Here's the one we saw before:

```
2: enp0s31f6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state
❶ UP group default qlen 1000
  ❷ link/ether 40:8d:5c:fc:24:1f brd ff:ff:ff:ff:ff:ff
    inet 10.23.2.4/24 brd 10.23.2.255 scope global noprefixroute enp0s31f6
       valid_lft forever preferred_lft forever
```

```
    inet6 2001:db8:8500:e:52b6:59cc:74e9:8b6e/64 scope global dynamic
noprefixroute
       valid_lft 86054sec preferred_lft 86054sec
    inet6 fe80::d05c:97f9:7be8:bca/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Each network interface gets a number; this one is 2. Interface 1 is almost always the loopback described in Section 9.16. The flag `UP` tells you that the interface is working ❶. In addition to the internet layer pieces that we've already covered, you also see the MAC address on the physical layer, `link/ether` ❷.

Although `ip` shows some hardware information, it's designed primarily for viewing and configuring the software layers attached to the interfaces. To dig deeper into the hardware and physical layer behind a network interface, use something like the `ethtool` command to display or change the settings on Ethernet cards. (We'll look briefly at wireless networks in Section 9.27.)

9.11 Introduction to Network Interface Configuration

You've now seen all of the basic elements that go into the lower levels of a network stack: the physical layer, the network (internet) layer, and the Linux kernel's network interfaces. In order to combine these pieces to connect a Linux machine to the internet, you or a piece of software must do the following:

1. Connect the network hardware and ensure that the kernel has a driver for it. If the driver is present, `ip address show` includes an entry for the device, even if it hasn't been configured.
2. Perform any additional physical layer setup, such as choosing a network name or password.
3. Assign IP address(es) and subnets to the kernel network interface so that the kernel's device drivers (physical layer) and internet subsystems (internet layer) can talk to each other.
4. Add any additional necessary routes, including the default gateway.

When all machines were big stationary boxes wired together, this was relatively straightforward: the kernel did step 1, you didn't need step 2, and you'd do step 3 with the old `ifconfig` command and step 4 with the old `route` command. We'll briefly look at how to do this with the `ip` command.

9.11.1 Manually Configuring Interfaces

We're now going to see how to set up interfaces manually, but we won't go into too much detail because doing so is rarely needed and error-prone. This is typically something you'd only do when experimenting with your system. Even when configuring, you may wish to use a tool such as Netplan to build the configuration in a text file instead of using a series of commands as shown next.

You can bind an interface to the internet layer with the `ip` command. To add an IP address and subnet for a kernel network interface, you'd do this:

```
# ip address add address/subnet dev interface
```

Here, *interface* is the name of the interface, such as *enp0s31f6* or *eth0*. This also works for IPv6, except that you need to add parameters (for example, to indicate link-local status). If you'd like to see all of the options, see the ip-address(8) manual page.

9.11.2 Manually Adding and Deleting Routes

With the interface up, you can add routes, which is typically just a matter of setting the default gateway, like this:

```
# ip route add default via gw-address dev interface
```

The *gw-address* parameter is the IP address of your default gateway; it *must* be an address in a locally connected subnet assigned to one of your network interfaces.

To remove the default gateway, run:

```
# ip route del default
```

You can easily override the default gateway with other routes. For example, say your machine is on subnet 10.23.2.0/24, you want to reach a subnet at 192.168.45.0/24, and you know that the host at 10.23.2.44 can act as a router for that subnet. Run this command to send traffic bound for 192.168.45.0 to that router:

```
# ip route add 192.168.45.0/24 via 10.23.2.44
```

You don't need to specify the router in order to delete a route:

```
# ip route del 192.168.45.0/24
```

Before you go crazy with routes, you should know that configuring routes is often more complicated than it appears. For this particular example, you also have to make sure that the routing for all hosts on 192.163.45.0/24 can lead back to 10.23.2.0/24, or the first route you add is basically useless.

Normally, you should keep things as simple as possible, setting up local networks so that their hosts need only a default route. If you need multiple subnets and the ability to route between them, it's usually best to configure the routers acting as the default gateways to do all of the work of routing between different local subnets. (You'll see an example in Section 9.21.)

9.12 Boot-Activated Network Configuration

We've discussed ways to manually configure a network, and the traditional way to ensure the correctness of a machine's network configuration was to have init run a script to run the manual configuration at boot time. This boils down to running a tool like `ip` somewhere in the chain of boot events.

There have been many attempts in Linux to standardize configuration files for boot-time networking. The tools `ifup` and `ifdown` are among them; for example, a boot script can (in theory) run `ifup eth0` to run the correct `ip` commands to set up an interface. Unfortunately, different distributions have completely different implementations of `ifup` and `ifdown`, and as a result, their configuration files are also different.

There is a deeper disparity due to the fact that network configuration elements are present in each of the different network layers; a consequence is that the software responsible for making networking happen is in several parts of kernel and user-space tools, written and maintained by different developers. In Linux, there is a general agreement not to share configuration files among separate tool suites or libraries, because changes made for one tool could break another.

Dealing with network configuration in several different places makes it difficult to manage systems. As a result, there are several different network management tools that each have their own approach to the configuration problem. However, these tend to be specialized for the particular kind of role that a Linux machine can serve. A tool might work on a desktop but not be appropriate for a server.

A tool called Netplan offers a different approach to the configuration problem. Rather than managing the network, Netplan is nothing more than a unified network configuration standard and a tool to transform that configuration into the files used by existing network managers. Currently, Netplan supports NetworkManager and systemd-networkd, which we'll talk about later in this chapter. Netplan files are in YAML format, and reside in */etc/netplan*.

Before we can talk about network configuration managers, let's look a little closer at some of the issues they face.

9.13 Problems with Manual and Boot-Activated Network Configuration

Although most systems used to configure the network in their boot mechanisms—and many servers still do—the dynamic nature of modern networks means that most machines don't have static (unchanging) IP addresses. In IPv4, rather than storing the IP address and other network information on your machine, your machine gets this information from somewhere on the local physical network when it first attaches to that network. Most normal network client applications don't particularly care what IP address your machine uses, as long as it works. Dynamic Host Configuration Protocol (DHCP,

described in Section 9.19) tools do the basic network layer configuration on typical IPv4 clients. In IPv6, clients are capable of configuring themselves to a certain extent; we'll look at that briefly in Section 9.20.

There's more to the story, though. For example, wireless networks add further dimensions to interface configuration, such as network names, authentication, and encryption techniques. When you step back to look at the bigger picture, you see that your system needs a way to answer the following questions:

- If the machine has multiple physical network interfaces (such as a notebook with wired and wireless Ethernet), how do you choose which one(s) to use?
- How should the machine set up the physical interface? For wireless networks, this includes scanning for network names, choosing a name, and negotiating authentication.
- Once the physical network interface is connected, how should the machine set up the software network layers, such as the internet layer?
- How can you let a user choose connectivity options? For example, how do you let a user choose a wireless network?
- What should the machine do if it loses connectivity on a network interface?

Answering these questions is usually more than simple boot scripts can handle, and it's a real hassle to do it all by hand. The answer is to use a system service that can monitor physical networks and choose (and automatically configure) the kernel network interfaces based on a set of rules that makes sense to the user. The service should also be able to respond to requests from users, who in turn should be able to change the wireless network they're on without having to become root.

9.14 Network Configuration Managers

There are several ways to automatically configure networks in Linux-based systems. The most widely used option on desktops and notebooks is NetworkManager. There is an add-on to systemd, called systemd-networkd, that can do basic network configuration and is useful for machines that don't need much flexibility (such as servers), but it doesn't have the dynamic capabilities of NetworkManager. Other network configuration management systems are mainly targeted for smaller embedded systems, such as OpenWRT's `netifd`, Android's ConnectivityManager service, ConnMan, and Wicd.

We'll briefly discuss NetworkManager because it's the one you're most likely to encounter. We won't go into a tremendous amount of detail, though, because after you see the basic concepts, NetworkManager and other configuration systems will be much easier to understand. If you're interested in systemd-networkd, the systemd.network(5) manual page describes the settings, and the configuration directory is */etc/systemd/network*.

9.14.1 NetworkManager Operation

NetworkManager is a daemon that the system starts upon boot. Like most daemons, it does not depend on a running desktop component. Its job is to listen to events from the system and users and to change the network configuration based on a set of rules.

When running, NetworkManager maintains two basic levels of configuration. The first is a collection of information about available hardware devices, which it normally collects from the kernel and maintains by monitoring udev over the Desktop Bus (D-Bus). The second configuration level is a more specific list of *connections*: hardware devices and additional physical and network layer configuration parameters. For example, a wireless network can be represented as a connection.

To activate a connection, NetworkManager often delegates the tasks to other specialized network tools and daemons, such as `dhclient`, to get internet layer configuration from a locally attached physical network. Because network configuration tools and schemes vary among distributions, NetworkManager uses plug-ins to interface with them, rather than imposing its own standard. There are plug-ins for the both the Debian/Ubuntu and Red Hat–style interface configuration, for example.

Upon startup, NetworkManager gathers all available network device information, searches its list of connections, and then decides to try to activate one. Here's how it makes that decision for Ethernet interfaces:

1. If a wired connection is available, try to connect using it. Otherwise, try the wireless connections.
2. Scan the list of available wireless networks. If a network is available that you've previously connected to, NetworkManager will try it again.
3. If more than one previously connected wireless network is available, select the most recently connected.

After establishing a connection, NetworkManager maintains it until the connection is lost, a better network becomes available (for example, you plug in a network cable while connected over wireless), or the user forces a change.

9.14.2 NetworkManager Interaction

Most users interact with NetworkManager through an applet on the desktop; it's usually an icon in the upper or lower right that indicates the connection status (wired, wireless, or not connected). When you click the icon, you get a number of connectivity options, such as a choice of wireless networks and an option to disconnect from your current network. Each desktop environment has its own version of this applet, so it looks a little different on each one.

In addition to the applet, there are a few tools that you can use to query and control NetworkManager from your shell. For a very quick summary of your current connection status, use the `nmcli` command with no arguments.

You'll get a list of interfaces and configuration parameters. In some ways, this is like `ip` except that there's more detail, especially when you're viewing wireless connections.

The `nmcli` command allows you to control NetworkManager from the command line. This is a somewhat extensive command; in fact, there's an nmcli-examples(5) manual page in addition to the usual nmcli(1) manual page.

Finally, the utility `nm-online` will tell you whether the network is up or down. If the network is up, the command returns `0` as its exit code; it's non-zero otherwise. (For more on how to use an exit code in a shell script, see Chapter 11.)

9.14.3 NetworkManager Configuration

NetworkManager's general configuration directory is usually */etc/NetworkManager*, and there are several different kinds of configuration. The general configuration file is *NetworkManager.conf*. The format is similar to the XDG-style *.desktop* and Microsoft *.ini* files, with key-value parameters falling into different sections. You'll find that nearly every configuration file has a `[main]` section that defines the plug-ins to use. Here's a simple example that activates the ifupdown plug-in used by Ubuntu and Debian:

```
[main]
plugins=ifupdown,keyfile
```

Other distribution-specific plug-ins are ifcfg-rh (for Red Hat–style distributions) and ifcfg-suse (for SuSE). The keyfile plug-in that you also see here supports NetworkManager's native configuration file support. When using the plug-in, you can see all of the system's known connections in */etc/NetworkManager/system-connections*.

For the most part, you won't need to change *NetworkManager.conf* because the more specific configuration options are found in other files.

Unmanaged Interfaces

Although you may want NetworkManager to manage most of your network interfaces, there will be times when you want it to ignore interfaces. For example, most users wouldn't need any kind of dynamic configuration on the localhost (*lo*; see Section 9.16) interface, because its configuration never changes. You also want to configure this interface early in the boot process, because basic system services often depend on it. Most distributions keep NetworkManager away from localhost.

You can tell NetworkManager to disregard an interface by using plug-ins. If you're using the ifupdown plug-in (for example, in Ubuntu and Debian), add the interface configuration to your */etc/network/interfaces* file and then set the value of `managed` to `false` in the `ifupdown` section of the *NetworkManager.conf* file:

```
[ifupdown]
managed=false
```

For the ifcfg-rh plug-in that Fedora and Red Hat use, look for a line like this in the */etc/sysconfig/network-scripts* directory that contains the *ifcfg-** configuration files:

```
NM_CONTROLLED=yes
```

If this line is not present or the value is set to `no`, NetworkManager ignores the interface. In the case of localhost, you'll find it deactivated in the *ifcfg-lo* file. You can also specify a hardware address to ignore, like this:

```
HWADDR=10:78:d2:eb:76:97
```

If you don't use either of these network configuration schemes, you can still use the keyfile plug-in to specify the unmanaged device directly inside your *NetworkManager.conf* file using its MAC address. Here's an example showing two unmanaged devices:

```
[keyfile]
unmanaged-devices=mac:10:78:d2:eb:76:97;mac:1c:65:9d:cc:ff:b9
```

Dispatching

One final detail of NetworkManager configuration relates to specifying additional system actions for when a network interface goes up or down. For example, some network daemons need to know when to start or stop listening on an interface in order to work correctly (such as the secure shell daemon discussed in the next chapter).

When a system's network interface status changes, NetworkManager runs everything in */etc/NetworkManager/dispatcher.d* with an argument such as `up` or `down`. This is relatively straightforward, but many distributions have their own network control scripts so they don't place the individual dispatcher scripts in this directory. Ubuntu, for example, has just one script named `01ifupdown` that runs everything in an appropriate subdirectory of */etc/network*, such as */etc/network/if-up.d*.

As with the rest of the NetworkManager configuration, the details of these scripts are relatively unimportant; all you need to know is how to track down the appropriate location if you need to make an addition or change (or use Netplan and let it figure out the location for you). As ever, don't be shy about looking at scripts on your system.

9.15 Resolving Hostnames

One of the final basic tasks in any network configuration is hostname resolution with DNS. You've already seen the `host` resolution tool that translates a name such as *www.example.com* to an IP address such as 10.23.2.132.

DNS differs from the network elements we've looked at so far because it's in the application layer, entirely in user space. Therefore, it's technically slightly out of place in this chapter alongside the internet and physical

layer discussion. However, without proper DNS configuration, your internet connection is practically worthless. No one in their right mind advertises IP addresses (much less IPv6 addresses) for websites and email addresses, because a host's IP address is subject to change and it's not easy to remember a bunch of numbers.

Practically all network applications on a Linux system perform DNS lookups. The resolution process typically unfolds like this:

1. The application calls a function to look up the IP address behind a hostname. This function is in the system's shared library, so the application doesn't need to know the details of how it works or whether the implementation will change.
2. When the function in the shared library runs, it acts according to a set of rules (found in */etc/nsswitch.conf*; see Section 9.15.4) to determine a plan of action on lookups. For example, the rules usually say that even before going to DNS, check for a manual override in the */etc/hosts* file.
3. When the function decides to use DNS for the name lookup, it consults an additional configuration file to find a DNS name server. The name server is given as an IP address.
4. The function sends a DNS lookup request (over the network) to the name server.
5. The name server replies with the IP address for the hostname, and the function returns this IP address to the application.

This is the simplified version. In a typical contemporary system, there are more actors attempting to speed up the transaction or add flexibility. Let's ignore that for now and look at some of the basic pieces. As with other kinds of network configuration, you probably won't need to change hostname resolution, but it's helpful to see how it works.

9.15.1 /etc/hosts

On most systems, you can override hostname lookups with the */etc/hosts* file. It usually looks like this:

```
127.0.0.1       localhost
10.23.2.3       atlantic.aem7.net       atlantic
10.23.2.4       pacific.aem7.net         pacific
::1             localhost ip6-localhost
```

You'll nearly always see the entry (or entries) for localhost here (see Section 9.16). The other entries here illustrate a simple way to add hosts on a local subnet.

NOTE *In the bad old days, there was one central* hosts *file that everyone copied to their own machine in order to stay up to date (see RFCs 606, 608, 623, and 625), but as the ARPANET/internet grew, this quickly got out of hand.*

9.15.2 resolv.conf

The traditional configuration file for DNS servers is */etc/resolv.conf*. When things were simpler, a typical example might have looked like this, where the ISP's name server addresses are 10.32.45.23 and 10.3.2.3:

```
search mydomain.example.com example.com
nameserver 10.32.45.23
nameserver 10.3.2.3
```

The `search` line defines rules for incomplete hostnames (just the first part of the hostname—for example, `myserver` instead of `myserver.example.com`). Here, the resolver library would try to look up *`host`*`.mydomain.example.com` and *`host`*`.example.com`.

Generally, name lookups are no longer this straightforward. Many enhancements and modifications have been made to the DNS configuration.

9.15.3 Caching and Zero-Configuration DNS

There are two main problems with the traditional DNS configuration. First, the local machine does not cache name server replies, so frequent repeated network access may be unnecessarily slow due to name server requests. To solve this problem, many machines (and routers, if acting as name servers) run an intermediate daemon to intercept name server requests and cache the reply, and then use the cached answers if possible. The most common of these daemons is systemd-resolved; you might also see dnsmasq or nscd on your system. You can also set up BIND (the standard Unix name server daemon) as a cache. You can often tell that you're running a name server caching daemon if you see 127.0.0.53 or 127.0.0.1 either in your */etc/resolv.conf* file or listed as the server when you run `nslookup -debug` *`host`*. Take a closer look, though. If you're running systemd-resolved, you might notice that */etc/resolv.conf* isn't even a file in */etc*; it's a link to an automatically generated file in */run*.

There's a lot more to systemd-resolved than meets the eye, as it can combine several name lookup services and expose them differently for each interface. This addresses the second problem with the traditional name server setup: it can be particularly inflexible if you want to be able to look up names on your local network without messing around with a lot of configuration. For example, if you set up a network appliance on your network, you'll want to be able to call it by name immediately. This is part of the idea behind zero-configuration name service systems such as Multicast DNS (mDNS) and Link-Local Multicast Name Resolution (LLMNR). If a process wants to find a host by name on the local network, it just broadcasts a request over the network; if present, the target host replies with its address. These protocols go beyond hostname resolution by also providing information about available services.

You can check the current DNS settings with the `resolvectl status` command (note that this might be called `systemd-resolve` on older systems). You'll get a list of global settings (typically of little use), and then you'll see the settings for each individual interface. It'll look like this:

```
Link 2 (enp0s31f6)
      Current Scopes: DNS
       LLMNR setting: yes
MulticastDNS setting: no
      DNSSEC setting: no
    DNSSEC supported: no
         DNS Servers: 8.8.8.8
          DNS Domain: ~.
```

You can see various supported name protocols here, as well as the name server that systemd-resolved consults for a name that it doesn't know.

We're not going to go further into DNS or systemd-resolved because it's such a vast topic. If you want to change your settings, take a look at the resolved.conf(5) manual page and proceed to change */etc/systemd/resolved.conf*. However, you'll probably need to read up on a lot of the systemd-resolved documentation, as well as get familiar with DNS in general from a source such as *DNS and BIND*, 5th edition, by Cricket Liu and Paul Albitz (O'Reilly, 2006).

9.15.4 /etc/nsswitch.conf

Before we leave the topic of name lookups, there's one last setting you should be aware of. The */etc/nsswitch.conf* file is the traditional interface for controlling several name-related precedence settings on your system, such as user and password information, and it has a host lookup setting. The file on your system should have a line like this:

```
hosts:          files dns
```

Putting `files` ahead of `dns` here ensures that, when looking up hosts, your system checks the */etc/hosts* file for host lookup before asking any DNS server, including systemd-resolved. This is usually a good idea (especially for looking up localhost, as discussed next), but your */etc/hosts* file should be as *short* as possible. Don't put anything in there to boost performance; doing so will burn you later. You can put hosts within a small private LAN in */etc/hosts*, but the general rule of thumb is that if a particular host has a DNS entry, it has no place in */etc/hosts*. (The */etc/hosts* file is also useful for resolving hostnames in the early stages of booting, when the network may not be available.)

All of this works through standard calls in the system library. It can be complicated to remember all of the places that name lookups can happen, but if you ever need to trace something from the bottom up, start with */etc/nsswitch.conf*.

9.16 Localhost

When running `ip address show`, you'll notice the *lo* interface:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
```

The *lo* interface is a virtual network interface called the *loopback* because it "loops back" to itself. The effect is that connecting to 127.0.0.1 (or ::1 in IPv6) is connecting to the machine that you're currently using. When outgoing data to localhost reaches the kernel network interface for *lo*, the kernel just repackages it as incoming data and sends it back through *lo*, for use by any server program that's listening (by default, most do).

The *lo* loopback interface is often the only place you might see static network configuration in boot-time scripts. For example, Ubuntu's `ifup` command reads */etc/network/interfaces*. However, this is often redundant, because systemd configures the loopback interface upon startup.

The loopback interface has one peculiarity, which you might have noticed. The netmask is /8, and anything starting with 127 is assigned to loopback. This allows you to run different servers on different IPv4 addresses in the loopback space without configuring additional interfaces. One server that takes advantage of this is systemd-resolved, which uses 127.0.0.53. This way, it won't interfere with another name server running on 127.0.0.1. So far, IPv6 defines only one loopback address, but there are proposals to change this.

9.17 The Transport Layer: TCP, UDP, and Services

So far, we've only seen how packets move from host to host on the internet—in other words, the *where* question from the beginning of the chapter. Now let's start to answer the question of *what* is transmitted. It's important to know how your computer presents the packet data it receives from other hosts to its running processes. It would be difficult and inconvenient for user-space programs to deal with a bunch of raw packets the way that the kernel does. Flexibility is especially important: more than one application should be able to talk to the network at the same time (for example, you might have email and several web clients running).

Transport layer protocols bridge the gap between the raw packets of the internet layer and the refined needs of applications. The two most popular transport protocols are the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP). We'll concentrate on TCP because it's by far the most common protocol in use, but we'll also take a quick look at UDP.

9.17.1 TCP Ports and Connections

TCP provides for multiple network applications on one machine by means of network *ports*, which are just numbers used in conjunction with an IP address. If an IP address of a host is like the postal address of an apartment building, a port number is like a mailbox number—it's a further subdivision.

When using TCP, an application opens a *connection* (not to be confused with NetworkManager connections) between one port on its own machine and a port on a remote host. For example, an application such as a web browser could open a connection between port 36406 on its own machine and port 80 on a remote host. From the application's point of view, port 36406 is the local port and port 80 is the remote port.

You can identify a connection by using the pair of IP addresses and port numbers. To view the connections currently open on your machine, use `netstat`. Here's an example that shows TCP connections; the `-n` option disables hostname resolution (DNS), and `-t` limits the output to TCP:

```
$ netstat -nt
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 10.23.2.4:47626         10.194.79.125:5222      ESTABLISHED
tcp        0      0 10.23.2.4:41475         172.19.52.144:6667      ESTABLISHED
tcp        0      0 10.23.2.4:57132         192.168.231.135:22      ESTABLISHED
```

The Local Address and Foreign Address fields refer to connections from your machine's point of view, so the machine here has an interface configured at 10.23.2.4, and ports 47626, 41475, and 57132 on the local side are all connected. The first connection here shows port 47626 connected to port 5222 of 10.194.79.125.

To show only IPv6 connections, add `-6` to the `netstat` options.

Establishing TCP Connections

To establish a transport layer connection, a process on one host initiates the connection from one of its local ports to a port on a second host with a special series of packets. In order to recognize the incoming connection and respond, the second host must have a process *listening* on the correct port. Usually, the connecting process is called the *client*, and the listener is called the *server* (more about this in Chapter 10).

The important thing to know about the ports is that the client picks a port on its side that isn't currently in use, and nearly always connects to some well-known port on the server side. Recall this output from the `netstat` command in the preceding section:

```
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 10.23.2.4:47626         10.194.79.125:5222      ESTABLISHED
```

With a little knowledge about port-numbering conventions, you can see that this connection was probably initiated by a local client to a remote server, because the port on the local side (47626) looks like a dynamically assigned number, whereas the remote port (5222) is a well-known service listed in */etc/services* (the Jabber or XMPP messaging service, to be specific). You'll see many connections to port 443 (the default for HTTPS) on most desktop machines.

NOTE *A dynamically assigned port is called an* ephemeral port.

However, if the local port in the output is well known, a remote host probably initiated the connection. In this example, remote host 172.24.54.234 has connected to port 443 on the local host:

```
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 10.23.2.4:443           172.24.54.234:43035     ESTABLISHED
```

A remote host connecting to your machine on a well-known port implies that a server on your local machine is listening on this port. To confirm this, list all TCP ports that your machine is listening on with `netstat`, this time with the `-l` option, which shows ports that processes are listening on:

```
$ netstat -ntl
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
❶ tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
❷ tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN
❸ tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN
--snip--
```

Line ❶ with 0.0.0.0:80 as the local address shows that the local machine is listening on port 80 for connections from any remote machine; it's the same for port 443 (line ❷). A server can restrict the access to certain interfaces, as shown in line ❸, where something is listening for connections only on the localhost interface. In this case, it's systemd-resolved; we talked about why it's listening using 127.0.0.53 instead of 127.0.0.1 back in Section 9.16. To learn even more, use `lsof` to identify the specific process that's listening (as discussed in Section 10.5.1).

Port Numbers and /etc/services

How do you know if a port is well known? There's no single way to tell, but a good place to start is to look in */etc/services*, which translates well-known port numbers into names. This is a plaintext file. You should see entries like this:

```
ssh             22/tcp              # SSH Remote Login Protocol
smtp            25/tcp
domain          53/udp
```

The first column is a name, and the second column indicates the port number and the specific transport layer protocol (which can be other than TCP).

NOTE *In addition to* /etc/services*, an online registry for ports at* http://www.iana.org/ *is governed by the RFC 6335 network standards document.*

On Linux, only processes running as the superuser can use ports 1 through 1023, also known as system, well-known, or privileged ports. All user processes may listen on and create connections from ports 1024 and up.

Characteristics of TCP

TCP is popular as a transport layer protocol because it requires relatively little from the application side. An application process only needs to know how to open (or listen for), read from, write to, and close a connection. To the application, it seems as if there are incoming and outgoing streams of data; the process is nearly as simple as working with a file.

However, there's a lot of work going on behind the scenes. For one, the TCP implementation needs to know how to break an outgoing data stream from a process into packets. The harder part, though, is knowing how to convert a series of incoming packets into an input data stream for processes to read, especially when incoming packets don't necessarily arrive in the correct order. In addition, a host using TCP must check for errors: packets can get lost or mangled when sent across the internet, and a TCP implementation must detect and correct these situations. Figure 9-4 shows a simplification of how a host might use TCP to send a message.

Luckily, you need to know next to nothing about this mess other than that the Linux TCP implementation is primarily in the kernel and that utilities that work with the transport layer tend to manipulate kernel data structures. One example is the iptables packet-filtering system discussed in Section 9.25.

9.17.2 UDP

UDP is a far simpler transport layer than TCP. It defines a transport only for single messages; there is no data stream. At the same time, unlike TCP, UDP won't correct for lost or out-of-order packets. In fact, although UDP has ports, it doesn't even have connections! One host simply sends a message from one of its ports to a port on a server, and the server sends something back if it wants to. However, UDP *does* have error detection for data inside a packet; a host can detect if a packet gets mangled, but it doesn't have to do anything about it.

Where TCP is like having a telephone conversation, UDP is like sending a letter, telegram, or instant message (except that instant messages are more reliable). Applications that use UDP are often concerned with speed—sending a message as quickly as possible. They don't want the overhead of TCP because they assume the network between two hosts is generally reliable. They don't need TCP's error correction because they either have their own error detection systems or simply don't care about errors.

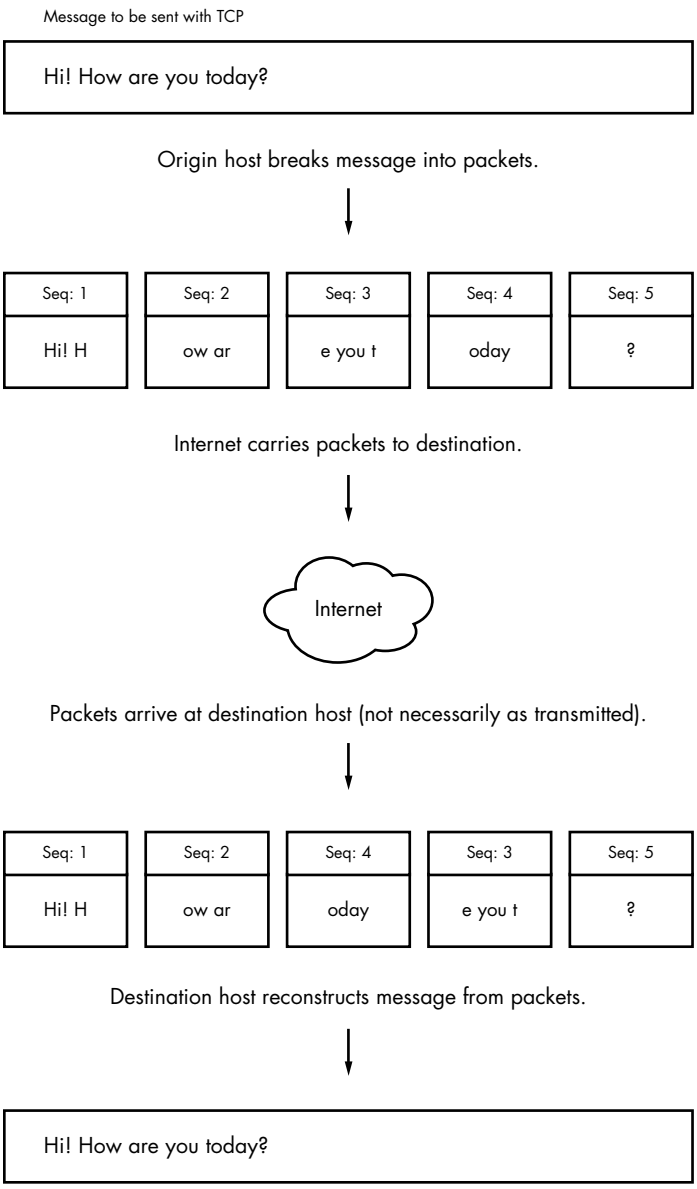


Figure 9-4: Sending a message with TCP

One example of an application that uses UDP is the *Network Time Protocol (NTP)*. A client sends a short and simple request to a server to get the current time, and the response from the server is equally brief. Because the client wants the response as quickly as possible, UDP suits the application; if the response from the server gets lost somewhere in the network, the client can just resend a request or give up. Another example is video chat. In this case, pictures are sent with UDP, and if some pieces get lost along the way, the client on the receiving end compensates the best it can.

NOTE *The rest of this chapter deals with more advanced networking topics, such as network filtering and routers, as they relate to the lower network layers that we've already seen: physical, network, and transport. If you like, feel free to skip ahead to the next chapter to see the application layer where everything comes together in user space. You'll see processes that actually use the network rather than just throwing around a bunch of addresses and packets.*

9.18 Revisiting a Simple Local Network

Now we'll look at additional components of the simple network introduced in Section 9.4. This network consists of one LAN as a subnet and a router that connects the subnet to the rest of the internet. You'll learn the following:

- How a host on the subnet automatically gets its network configuration
- How to set up routing
- What a router really is
- How to know which IP addresses to use for the subnet
- How to set up firewalls to filter out unwanted traffic from the internet

For the most part, we'll concentrate on IPv4 (if for no other reason than that the addresses are easier to read), but when IPv6 differs, you'll see how.

Let's start by looking at how a host on the subnet automatically gets its network configuration.

9.19 Understanding DHCP

Under IPv4, when you set a network host to get its configuration automatically from the network, you're telling it to use the Dynamic Host Configuration Protocol (DHCP) to get an IP address, subnet mask, default gateway, and DNS servers. Aside from not having to enter these parameters by hand, network administrators gain other advantages with DHCP, such as preventing IP address clashes and minimizing the impact of network changes. It's very rare to see a network that doesn't use DHCP.

For a host to get its configuration with DHCP, it must be able to send messages to a DHCP server on its connected network. Therefore, each physical network should have its own DHCP server, and on a simple network (such as the one in Section 9.1), the router usually acts as the DHCP server.

NOTE *When making an initial DHCP request, a host doesn't even know the address of a DHCP server, so it broadcasts the request to all hosts (usually all hosts on its physical network).*

When a machine asks a DHCP server to assign it an IP address, it's really asking for a *lease* on an address for a certain amount of time. When the lease is up, a client can ask to renew the lease.

9.19.1 Linux DHCP Clients

Although there are many different kinds of network manager systems, there are only two DHCP clients that do the actual work of obtaining leases. The traditional standard client is the Internet Software Consortium (ISC) `dhclient` program. However, systemd-networkd now also includes a built-in DHCP client.

Upon startup, `dhclient` stores its process ID in */var/run/dhclient.pid* and its lease information in */var/lib/dhcp/dhclient.leases*.

You can test `dhclient` by hand on the command line, but before doing so you *must* remove any default gateway route (see Section 9.11.2). To run the test, simply specify the network interface name (here, it's enp0s31f6):

```
# dhclient enp0s31f6
```

Unlike `dhclient`, the systemd-networkd DHCP client can't be run by hand on the command line. The configuration, described the systemd .network(5) manual page, is in */etc/systemd/network*, but like other kinds of network configuration, can be automatically generated by Netplan.

9.19.2 Linux DHCP Servers

You can task a Linux machine with running a DHCP server, which provides a good amount of control over the addresses it gives out. However, unless you're administering a large network with many subnets, you're probably better off using specialized router hardware that includes built-in DHCP servers.

Probably the most important thing to know about DHCP servers is that you want only one running on the same subnet in order to avoid problems with clashing IP addresses or incorrect configurations.

9.20 Automatic IPv6 Network Configuration

DHCP works acceptably well in practice, but it relies on certain assumptions, including that there will be a DHCP server available, that the server is correctly implemented and stable, and that it can track and maintain leases. Although there's a version of DHCP for IPv6 called DHCPv6, there's an alternative that's far more common.

The IETF took advantage of the large IPv6 address space to devise a new way of network configuration that does not require a central server. This is called *stateless configuration*, because clients don't need to store any data such as lease assignments.

Stateless IPv6 network configuration starts with the link-local network. Recall that this network includes the addresses prefixed fe80::/64. Because there are so many available addresses on the link-local network, a host can generate an address that is unlikely to be duplicated anywhere on the network. Furthermore, the network prefix is already fixed, so the host can broadcast to the network, asking if any other host on the network is using the address.

Once the host has a link-local address, it can determine a global address. It does so by listening for a router advertisement (RA) message that routers occasionally send on the link-local network. The RA message includes the global network prefix, the router IP address, and possibly DNS information. With that information, the host can attempt to fill in the interface ID part of the global address, similar to what it did with the link-local address.

Stateless configuration relies on a global network prefix at most 64 bits long (in other words, its netmask is /64 or lower).

NOTE *Routers also send RA messages in response to router solicitation messages from hosts. These, as well as a few other messages, are part of the ICMP protocol for IPv6 (ICMPv6).*

9.21 Configuring Linux as a Router

Routers are just computers with more than one physical network interface. You can easily configure a Linux machine to be a router.

Let's look at an example. Say you have two LAN subnets, 10.23.2.0/24 and 192.168.45.0/24. To connect them, you have a Linux router machine with three network interfaces: two for the LAN subnets and one for an internet uplink, as shown in Figure 9-5.

As you can see, this doesn't look very different from the simple network example used in the rest of this chapter. The router's IP addresses for the LAN subnets are 10.23.2.1 and 192.168.45.1. When those addresses are configured, the routing table looks something like this (the interface names might vary in practice; ignore the internet uplink for now):

```
# ip route show
10.23.2.0/24 dev enp0s31f6 proto kernel scope link src 10.23.2.1 metric 100
192.168.45.0/24 dev enp0s1 proto kernel scope link src 192.168.45.1 metric 100
```

Now let's say that the hosts on each subnet have the router as their default gateway (10.23.2.1 for 10.23.2.0/24 and 192.168.45.1 for 192.168.45.0/24). If 10.23.2.4 wants to send a packet to anything outside of 10.23.2.0/24, it passes the packet to 10.23.2.1. For example, to send a packet from 10.23.2.4 (Host A) to 192.168.45.61 (Host E), the packet goes to 10.23.2.1 (the router) via its enp0s31f6 interface, then back out through the router's enp0s1 interface.

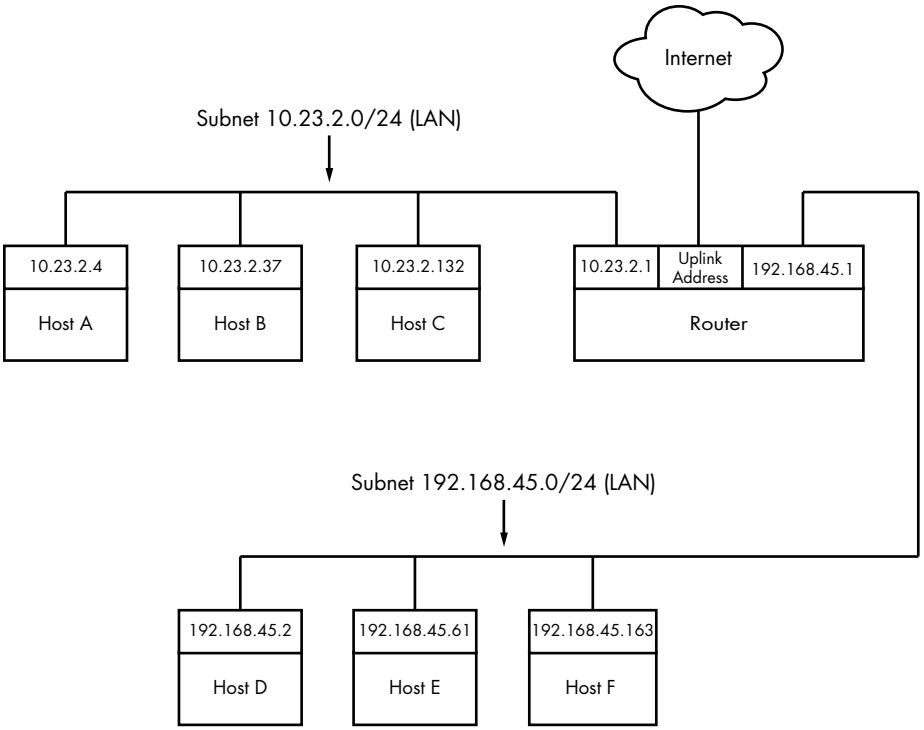


Figure 9-5: Two subnets joined with a router

However, in some basic configurations, the Linux kernel does not automatically move packets from one subnet to another. To enable this basic routing function, you need to enable *IP forwarding* in the router's kernel with this command:

```
# sysctl -w net.ipv4.ip_forward=1
```

As soon as you enter this command, the machine should start routing packets between subnets, assuming that the hosts on those subnets know to send their packets to the router you just created.

NOTE *You can check the status of IP forwarding with the* `sysctl net.ipv4.ip_forward` *command.*

To make this change permanent upon reboot, you can add it to your */etc/sysctl.conf* file. Depending on your distribution, you may have the option to put it into a file in */etc/sysctl.d* so that distribution updates won't overwrite your changes.

When the router also has the third network interface with an internet uplink, this same setup allows internet access for all hosts on both subnets because they're configured to use the router as the default gateway. But that's where things get more complicated. The problem is that certain IPv4

addresses such as 10.23.2.4 are not actually visible to the whole internet; they're on so-called private networks. To provide for internet connectivity, you must set up a feature called *Network Address Translation (NAT)* on the router. The software on nearly all specialized routers does this, so there's nothing out of the ordinary here, but let's examine the problem of private networks in a bit more detail.

9.22 Private Networks (IPv4)

Say you decide to build your own network. You have your machines, router, and network hardware ready. Given what you know about a simple network so far, your next question is, "What IP subnet should I use?"

If you want a block of internet addresses that every host on the internet can see, you can buy one from your ISP. However, because the range of IPv4 addresses is very limited, this costs a lot and isn't useful for much more than running a server that the rest of the internet can see. Most people don't really need this kind of service because they access the internet as a client.

The conventional, inexpensive alternative is to pick a private subnet from the addresses in the RFC 1918/6761 internet standards documents, shown in Table 9-2.

Table 9-2: Private Networks Defined by RFC 1918 and 6761

Network	Subnet mask	CIDR form
10.0.0.0	255.0.0.0	10.0.0.0/8
192.168.0.0	255.255.0.0	192.168.0.0/16
172.16.0.0	255.240.0.0	172.16.0.0/12

You can carve up private subnets as you wish. Unless you plan to have more than 254 hosts on a single network, pick a small subnet like 10.23.2.0/24, as we've been using throughout this chapter. (Networks with this netmask are sometimes called *class C* subnets. Although the term is technically obsolete, it's still useful.)

What's the catch? Hosts on the real internet know nothing about private subnets and won't send packets to them, so without some help, hosts on private subnets can't talk to the outside world. A router connected to the internet (with a true, nonprivate address) needs to have some way to fill in the gap between that connection and the hosts on a private network.

9.23 Network Address Translation (IP Masquerading)

NAT is the most commonly used way to share a single IP address with a private network, and it's nearly universal in home and small office networks. In Linux, the variant of NAT that most people use is known as *IP masquerading*.

The basic idea behind NAT is that the router doesn't just move packets from one subnet to another; it transforms them as it moves them. Hosts

on the internet know how to connect to the router, but they know nothing about the private network behind it. The hosts on the private network need no special configuration; the router is their default gateway.

The system works roughly like this:

1. A host on the internal private network wants to make a connection to the outside world, so it sends its connection request packets through the router.
2. The router intercepts the connection request packet rather than passing it out to the internet (where it would get lost because the public internet knows nothing about private networks).
3. The router determines the destination of the connection request packet and opens its own connection to the destination.
4. When the router obtains the connection, it fakes a "connection established" message back to the original internal host.
5. The router is now the middleman between the internal host and the destination. The destination knows nothing about the internal host; the connection on the remote host looks like it came from the router.

This isn't quite as simple as it sounds. Normal IP routing knows only source and destination IP addresses in the internet layer. However, if the router dealt only with the internet layer, each host on the internal network could establish only one connection to a single destination at a time (among other limitations), because there is no information in the internet layer part of a packet to distinguish among multiple requests from the same host to the same destination. Therefore, NAT must go beyond the internet layer and dissect packets to pull out more identifying information, particularly the UDP and TCP port numbers from the transport layers. UDP is fairly easy because there are ports but no connections, but the TCP transport layer is complex.

In order to set up a Linux machine to perform as a NAT router, you must activate all of the following inside the kernel configuration: network packet filtering ("firewall support"), connection tracking, iptables support, full NAT, and MASQUERADE target support. Most distribution kernels come with this support.

Next you need to run some complex-looking `iptables` commands to make the router perform NAT for its private subnet. Here's an example that applies to an internal Ethernet network on enp0s2 sharing an external connection at enp0s31f6 (you'll learn more about the `iptables` syntax in Section 9.25):

```
# sysctl -w net.ipv4.ip_forward=1
# iptables -P FORWARD DROP
# iptables -t nat -A POSTROUTING -o enp0s31f6 -j MASQUERADE
# iptables -A FORWARD -i enp0s31f6 -o enp0s2 -m state --state
ESTABLISHED,RELATED -j ACCEPT
# iptables -A FORWARD -i enp0s2 -o enp0s31f6 -j ACCEPT
```

You likely won't ever need to manually enter these commands unless you're developing your own software, especially with so much special-purpose router hardware available. However, a variety of virtualization software can set up NAT for use in networking for virtual machines and containers.

Although NAT works well in practice, remember that it's essentially a hack that extends the lifetime of the IPv4 address space. IPv6 does not need NAT, thanks to its larger and more sophisticated address space described in Section 9.7.

9.24 Routers and Linux

In the early days of broadband, users with less demanding needs simply connected their machine directly to the internet. But it didn't take long for many users to want to share a single broadband connection with their own networks, and Linux users in particular would often set up an extra machine to use as a router running NAT.

Manufacturers responded to this new market by offering specialized router hardware consisting of an efficient processor, some flash memory, and several network ports—with enough power to manage a typical simple network, run important software such as a DHCP server, and use NAT. When it came to software, many manufacturers turned to Linux to power their routers. They added the necessary kernel features, stripped down the user-space software, and created GUI-based administration interfaces.

Almost as soon as the first of these routers appeared, many people became interested in digging deeper into the hardware. One manufacturer, Linksys, was required to release the source code for its software under the terms of the license of one of its components, and soon specialized Linux distributions such as OpenWRT appeared for routers. (The "WRT" in these names came from the Linksys model number.)

Aside from the hobbyist aspect, there are good reasons to install these distributions on routers. They're often more stable than the manufacturer firmware, especially on older router hardware, and they typically offer additional features. For example, to bridge a network with a wireless connection, many manufacturers require you to buy matching hardware, but with OpenWRT installed, the manufacturer and age of the hardware don't really matter. This is because you're using a truly open operating system on the router that doesn't care what hardware you use as long as your hardware is supported.

You can use much of the knowledge in this book to examine the internals of custom Linux firmware, though you'll encounter differences, especially when logging in. As with many embedded systems, open firmware tends to use BusyBox to provide many shell features. BusyBox is a single executable program that offers limited functionality for many Unix commands such as the shell, `ls`, `grep`, `cat`, and `more`. (This saves a significant amount of memory.) In addition, the boot-time init tends to be very simple

on embedded systems. However, you typically won't find these limitations to be a problem, because custom Linux firmware often includes a web administration interface similar to what you'd see from a manufacturer.

9.25 Firewalls

Routers should always include some kind of firewall to keep undesirable traffic out of your network. A *firewall* is a software and/or hardware configuration that usually sits on a router between the internet and a smaller network, attempting to ensure that nothing "bad" from the internet harms the smaller network. You can also set up firewall features on any host to screen all incoming and outgoing data at the packet level (as opposed to at the application layer, where server programs usually try to perform some access control of their own). Firewalling on individual machines is sometimes called *IP filtering*.

A system can filter packets when it receives a packet, sends a packet, or forwards (routes) a packet to another host or gateway.

With no firewalling in place, a system just processes packets and sends them on their way. Firewalls put checkpoints for packets at the points of data transfer just identified. The checkpoints drop, reject, or accept packets, usually based on some of these criteria:

- The source or destination IP address or subnet
- The source or destination port (in the transport layer information)
- The firewall's network interface

Firewalls provide an opportunity to work with the subsystem of the Linux kernel that processes IP packets. Let's look at that now.

9.25.1 Linux Firewall Basics

In Linux, you create firewall rules in a series known as a *chain*. A set of chains makes up a *table*. As a packet moves through the various parts of the Linux networking subsystem, the kernel applies the rules in certain chains to the packets. For example, a new packet arriving from the physical layer is classified by the kernel as "input," so it activates rules in chains corresponding to input.

All of these data structures are maintained by the kernel. The whole system is called *iptables*, with an `iptables` user-space command to create and manipulate the rules.

NOTE *There's a newer system called* nftables *that is meant to replace iptables, but as of this writing, iptables is still the most widely used system. The command to administer nftables is* `nft`*, and there's an iptables-to-nftables translator called* `iptables-translate` *for the iptables commands shown in this book. To make things even more complicated, a system called* bpfilter *has been recently introduced with a different approach. Try not to get bogged down with the specifics of commands—it's the effects that matter.*

Because there can be many tables—each with its own sets of chains, which in turn can contain many rules—packet flow can become quite complicated. However, you'll normally work primarily with a single table named *filter* that controls basic packet flow. There are three basic chains in the filter table: INPUT for incoming packets, OUTPUT for outgoing packets, and FORWARD for routed packets.

Figures 9-6 and 9-7 show simplified flowcharts for where rules are applied to packets in the filter table. There are two figures because packets can either come into the system from a network interface (Figure 9-6) or be generated by a local process (Figure 9-7).

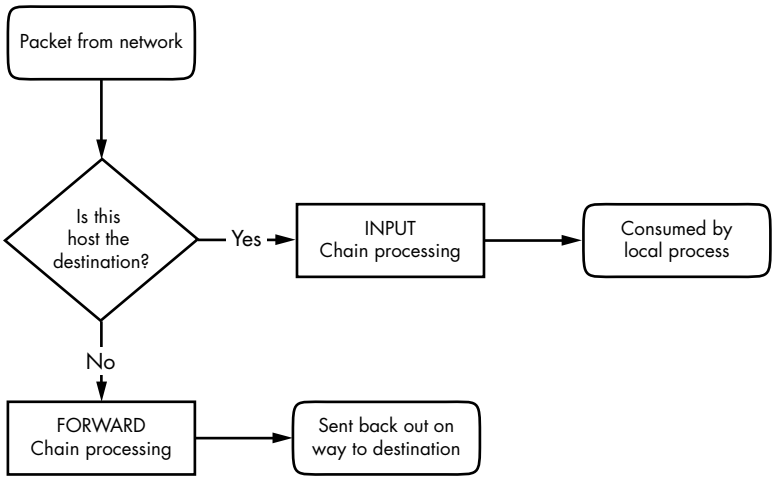


Figure 9-6: Chain-processing sequence for incoming packets from a network

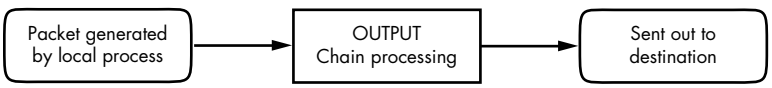


Figure 9-7: Chain-processing sequence for incoming packets from a local process

As you can see, an incoming packet from the network can be consumed by a user process and may not reach the FORWARD chain or the OUTPUT chain. Packets generated by user processes won't reach the INPUT or FORWARD chains.

This gets more complicated because there are many steps along the way other than just these three chains. For example, packets are subject to PREROUTING and POSTROUTING chains, and chain processing can also occur at any of the three lower network levels. For a big diagram of everything that's going on, search the internet for "Linux netfilter packet flow," but remember that these diagrams try to include every possible scenario for packet input and flow. It often helps to break the diagrams down by packet source, as in Figures 9-6 and 9-7.

9.25.2 Setting Firewall Rules

Let's look at how the iptables system works in practice. Start by viewing the current configuration with this command:

```
# iptables -L
```

The output is usually an empty set of chains, as follows:

```
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Each firewall chain has a default *policy* that specifies what to do with a packet if no rule matches the packet. The policy for all three chains in this example is ACCEPT, meaning that the kernel allows the packet to pass through the packet-filtering system. The DROP policy tells the kernel to discard the packet. To set the policy on a chain, use `iptables -P` like this:

```
# iptables -P FORWARD DROP
```

WARNING *Don't do anything rash with the policies on your machine until you've read through the rest of this section.*

Say that someone at 192.168.34.63 is annoying you. To prevent them from talking to your machine, run this command:

```
# iptables -A INPUT -s 192.168.34.63 -j DROP
```

The `-A INPUT` parameter appends a rule to the INPUT chain. The `-s 192.168.34.63` part specifies the source IP address in the rule, and `-j DROP` tells the kernel to discard any packet matching the rule. Therefore, your machine will throw out any packet coming from 192.168.34.63.

To see the rule in place, run **`iptables -L`** again:

```
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.34.63        anywhere
```

Unfortunately, your friend at 192.168.34.63 has told everyone on his subnet to open connections to your SMTP port (TCP port 25). To get rid of that traffic as well, run:

```
# iptables -A INPUT -s 192.168.34.0/24 -p tcp --destination-port 25 -j DROP
```

This example adds a netmask qualifier to the source address as well as `-p tcp` to specify TCP packets only. A further restriction, `--destination-port 25`, says that the rule should apply only to traffic to port 25. The IP table list for INPUT now looks like this:

```
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.34.63        anywhere
DROP       tcp  --  192.168.34.0/24      anywhere           tcp dpt:smtp
```

All is well until you hear from someone you know at 192.168.34.37 saying that she can't send you email because you blocked her machine. Thinking this is a quick fix, you run this command:

```
# iptables -A INPUT -s 192.168.34.37 -j ACCEPT
```

However, it doesn't work. To see why, look at the new chain:

```
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       all  --  192.168.34.63        anywhere
DROP       tcp  --  192.168.34.0/24      anywhere           tcp dpt:smtp
ACCEPT     all  --  192.168.34.37        anywhere
```

The kernel reads the chain from top to bottom, using the first rule that matches.

The first rule does not match 192.168.34.37, but the second does, because it applies to all hosts from 192.168.34.1 to 192.168.34.254 and this second rule says to drop packets. When a rule matches, the kernel carries out the action and looks no further down in the chain. (You might notice that 192.168.34.37 can send packets to any port on your machine *except* port 25 because the second rule applies *only* to port 25.)

The solution is to move the third rule to the top. First, delete the third rule with this command:

```
# iptables -D INPUT 3
```

Then *insert* that rule at the top of the chain with `iptables -I`:

```
# iptables -I INPUT -s 192.168.34.37 -j ACCEPT
```

To insert a rule elsewhere in a chain, put the rule number after the chain name (for example, `iptables -I INPUT 4 ...`).

9.25.3 Firewall Strategies

Although the preceding tutorial showed you how to insert rules and how the kernel processes IP chains, you haven't seen firewall strategies that actually work. Let's talk about that now.

There are two basic kinds of firewall scenarios: one for protecting individual machines (where you set rules in each machine's INPUT chain) and one for protecting a network of machines (where you set rules in the router's FORWARD chain). In both cases, you can't have serious security if you use a default policy of ACCEPT and continuously insert rules to drop packets from sources that start to send bad stuff. You must allow only the packets that you trust, and deny everything else.

For example, say your machine has an SSH server on TCP port 22. There's no reason for any random host to initiate a connection to any other port on your machine, and you shouldn't give any such host a chance. To set that up, first set the INPUT chain policy to DROP:

```
# iptables -P INPUT DROP
```

To enable ICMP traffic (for `ping` and other utilities), use this line:

```
# iptables -A INPUT -p icmp -j ACCEPT
```

Make sure that you can receive packets you send to both your own network IP address and 127.0.0.1 (localhost). Assuming your host's IP address is *my_addr*, do this:

```
# iptables -A INPUT -s 127.0.0.1 -j ACCEPT
# iptables -A INPUT -s my_addr -j ACCEPT
```

WARNING *Don't run these commands one by one on a machine to which you only have remote access. The very first `DROP` command will instantly block your access, and you won't be able to regain access until you intervene (for example, by rebooting the machine).*

If you control your entire subnet (and trust everything on it), you can replace *my_addr* with your subnet address and subnet mask—for example, `10.23.2.0/24`.

Now, although you still want to deny incoming TCP connections, you still need to make sure that your host can make TCP connections to the outside world. Because all TCP connections start with a SYN (connection request) packet, if you let all TCP packets through that aren't SYN packets, you're still okay:

```
# iptables -A INPUT -p tcp '!' --syn -j ACCEPT
```

The `!` symbol indicates a negation, so `! --syn` matches any non-SYN packet.

Next, if you're using remote UDP-based DNS, you must accept traffic from your name server so that your machine can look up names with DNS. Do this for *all* DNS servers in */etc/resolv.conf*. Use this command (where the name server's address is *ns_addr*):

```
# iptables -A INPUT -p udp --source-port 53 -s ns_addr -j ACCEPT
```

And finally, allow SSH connections from anywhere:

```
# iptables -A INPUT -p tcp --destination-port 22 -j ACCEPT
```

The preceding iptables settings work for many situations, including any direct connection (especially broadband) where an intruder is much more likely to port-scan your machine. You could also adapt these settings for a firewalling router by using the FORWARD chain instead of INPUT and using source and destination subnets where appropriate. For more advanced configurations, you may find a configuration tool such as Shorewall to be helpful.

This discussion has only touched on security policy. Remember that the key idea is to permit only the things that you find acceptable, not to try to find and exclude the bad stuff. Furthermore, IP firewalling is only one piece of the security picture. (You'll see more in the next chapter.)

9.26 Ethernet, IP, ARP, and NDP

There is one basic detail in the implementation of IP over Ethernet that we have yet to cover. Recall that a host must place an IP packet inside an Ethernet frame in order to transmit the packet across the physical layer to another host. Recall, too, that frames themselves do not include IP address information; they use MAC (hardware) addresses. The question is this: When constructing the Ethernet frame for an IP packet, how does the host know which MAC address corresponds to the destination IP address?

We don't normally think about this question much because networking software includes an automatic system of looking up MAC addresses. In IPv4, this is called *Address Resolution Protocol (ARP)*. A host using Ethernet as its physical layer and IP as the network layer maintains a small table called an *ARP cache* that maps IP addresses to MAC addresses. In Linux, the ARP cache is in the kernel. To view your machine's ARP cache, use the `ip neigh` command. (The "neigh" part will make sense when you see the IPv6 equivalent. The old command for working with the ARP cache is `arp`.)

```
$ ip -4 neigh
10.1.2.57 dev enp0s31f6 lladdr 1c:f2:9a:1e:88:fb REACHABLE
10.1.2.141 dev enp0s31f6 lladdr 00:11:32:0d:ca:82 STALE
10.1.2.1 dev enp0s31f6 lladdr 24:05:88:00:ca:a5 REACHABLE
```

We're using the `-4` option to restrict the output to IPv4. You can see the IP and hardware addresses for the hosts that the kernel knows about. The last field indicates the status of the entry in the cache. REACHABLE means that some communication with the host occurred recently, and STALE means that it's been a while, and the entry should be refreshed.

When a machine boots, its ARP cache is empty. So how do these MAC addresses get in the cache? It all starts when the machine wants to send a packet to another host. If a target IP address is not in an ARP cache, the following steps occur:

1. The origin host creates a special Ethernet frame containing an ARP request packet for the MAC address that corresponds to the target IP address.
2. The origin host broadcasts this frame to the entire physical network for the target's subnet.
3. If one of the other hosts on the subnet knows the correct MAC address, it creates a reply packet and frame containing the address and sends it back to the origin. Often, the host that replies *is* the target host and is simply replying with its own MAC address.
4. The origin host adds the IP-MAC address pair to the ARP cache and can proceed.

NOTE *Remember that ARP applies only to machines on local subnets. To reach destinations outside your subnet, your host sends the packet to the router, and it's someone else's problem after that. Of course, your host still needs to know the MAC address for the router, and it can use ARP to find it.*

The only real problem you can have with ARP is that your system's cache can get out of date if you're moving an IP address from one network interface card to another because the cards have different MAC addresses (for example, when testing a machine). Unix systems invalidate ARP cache entries if there's no activity after a while, so there shouldn't be any trouble other than a small delay for invalidated data, but you can delete an ARP cache entry immediately with this command:

```
# ip neigh del host dev interface
```

The ip-neighbour(8) manual page explains how to manually set ARP cache entries, but you shouldn't need to do this. Note the spelling.

NOTE *Don't confuse ARP with* Reverse Address Resolution Protocol (RARP). *RARP transforms a MAC address back to a hostname or IP address. Before DHCP became popular, some diskless workstations and other devices used RARP to get their configuration, but RARP is rare today.*

IPV6: NDP

You might be wondering why the commands manipulating the ARP cache don't contain "arp" (or, if you've vaguely seen this stuff before, you might wonder why we aren't using `arp`). In IPv6, there's a new mechanism called *Neighbor Discovery Protocol (NDP)* used on the link-local network. The `ip` command unifies ARP from IPv4 and NDP from IPv6. NDP includes these two kinds of messages:

Neighbor solicitation Used to obtain information about a link-local host, including the hardware address of the host.

Neighbor advertisement Used to respond to a neighbor solicitation message.

There are several other components of NDP, including the RA messages that you saw in Section 9.20.

9.27 Wireless Ethernet

In principle, wireless Ethernet ("Wi-Fi") networks aren't much different from wired networks. Much like any wired hardware, they have MAC addresses and use Ethernet frames to transmit and receive data, and as a result the Linux kernel can talk to a wireless network interface much as it would a wired network interface. Everything at the network layer and above is the same; the main differences are additional components in the physical layer, such as frequencies, network IDs, and security features.

Unlike wired network hardware, which is very good at automatically adjusting to nuances in the physical setup without much fuss, wireless network configuration is much more open-ended. To get a wireless interface working properly, Linux needs additional configuration tools.

Let's take a quick look at the additional components of wireless networks.

Transmission details These are physical characteristics, such as the radio frequency.

Network identification Because more than one wireless network can share the same basic medium, you have to be able to distinguish between them. The Service Set Identifier (SSID, also known as the "network name") is the wireless network identifier.

Management Although it's possible to configure wireless networking to have hosts talk directly to each other, most wireless networks are managed by one or more *access points* that all traffic goes through. Access points often bridge a wireless network with a wired network, making both appear as one single network.

Authentication You may want to restrict access to a wireless network. To do so, you can configure access points to require a password or other authentication key before they'll even talk to a client.

Encryption In addition to restricting the initial access to a wireless network, you normally want to encrypt all traffic that goes out across radio waves.

The Linux configuration and utilities that handle these components are spread out over a number of areas. Some are in the kernel; Linux features a set of wireless extensions that standardize user-space access to hardware. As far as user space goes, wireless configuration can get complicated, so most people prefer to use GUI frontends, such as the desktop applet for NetworkManager, to get things working. Still, it's worth looking at a few of the things happening behind the scenes.

9.27.1 iw

You can view and change kernel space device and network configuration with a utility called `iw`. To use `iw`, you normally need to know the network interface name for the device, such as *wlp1s0* (predictable device name) or *wlan0* (traditional name). Here's an example that dumps a scan of available wireless networks. (Expect a lot of output if you're in an urban area.)

```
# iw dev wlp1s0 scan
```

NOTE *The network interface must be up for this command to work (if it's not, run `ifconfig wlp1s0 up`), but because this is still in the physical layer, you don't need to configure any network layer parameters, such as an IP address.*

If the network interface has joined a wireless network, you can view the network details like this:

```
# iw dev wlp1s0 link
```

The MAC address in the output of this command is from the access point that you're currently talking to.

NOTE *The `iw` command distinguishes between physical device names (such as phy0) and network interface names (such as wlp1s0) and allows you to change various settings for each. You can even create more than one network interface for a single physical device. However, in nearly all basic cases, you'll just use the network interface name.*

Use `iw` to connect a network interface to an unsecured wireless network as follows:

```
# iw wlp1s0 connect network_name
```

Connecting to secured networks is a different story. For the rather insecure Wired Equivalent Privacy (WEP) system, you can use the `keys` parameter with the `iw connect` command. However, you shouldn't use WEP because it's not secure, and you won't find many networks that support it.

9.27.2 Wireless Security

For most wireless security setups, Linux relies on the wpa_supplicant daemon to manage both authentication and encryption for a wireless network interface. This daemon can handle the WPA2 and WPA3 (WiFi Protected Access; don't use the older, insecure WPA) schemes of authentication, as well as nearly any kind of encryption technique used on wireless networks. When the daemon first starts, it reads a configuration file (by default, */etc/wpa_supplicant.conf*) and attempts to identify itself to an access point and establish communication based on a given network name. The system is well documented; in particular, the wpa_supplicant(8) manual page is very detailed.

Running the daemon by hand every time you want to establish a connection is a lot of work. In fact, just creating the configuration file is tedious due to the number of possible options. To make matters worse, all of the work of running `iw` and wpa_supplicant simply allows your system to join a wireless physical network; it doesn't even set up the network layer. And that's where automatic network configuration managers such as NetworkManager take a lot of pain out of the process. Although they don't do any of the work on their own, they know the correct sequence and required configuration for each step toward getting a wireless network operational.

9.28 Summary

As you've seen, understanding the positions and roles of the various network layers is critical to understanding how Linux networking operates and how to perform network configuration. Although we've covered only the basics, more advanced topics in the physical, network, and transport layers are similar to what you've seen here. Layers themselves are often subdivided, as you just saw with the various pieces of the physical layer in a wireless network.

A substantial amount of action that you've seen in this chapter happens in the kernel, with some basic user-space control utilities to manipulate the kernel's internal data structures (such as routing tables). This is the traditional way of working with the network. However, as with many of the topics discussed in this book, some tasks aren't suitable for the kernel due to their complexity and need for flexibility, and that's where user-space utilities take over. In particular, NetworkManager monitors and queries the kernel and then manipulates the kernel configuration. Another example is support for dynamic routing protocols such as Border Gateway Protocol (BGP), which is used in large internet routers.

But you're probably a little bit bored with network configuration by now. Let's turn to *using* the network—the application layer.